Poise: An AI-Based Interview Practice and Feedback System

21CSC354T-FULL STACK WEB DEVELOPMENT

A MINI-PROJECT REPORT

Submitted By

KA Dharan Balaji [RA2411003020260]

Abishek V.P.T [RA2411003020266]

V Varadharajan [RA2411003020267]

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

of

FACULTY OF ENGINEERING AND TECHNOLOGY



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

RAMAPURAM CAMPUS, CHENNAI-600089

OCTOBER 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University U/S 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this mini project report titled **"POISE: AN AI-BASED INTERVIEW PRACTICE AND FEEDBACK SYSTEM"** is the bonafide work of **KA DHARAN BALAJI (RA2411003020260), ABISHEK V.P.T (RA2411003020266), and V VARADHARAJAN (RA2411003020267)**, students of **Third Year, Computer Science and Engineering**.

SIGNATURE

Dr. C.SHANMUGANATHAN, M.E., Ph.D.,

Assistant Professor,

Computer Science and Engineering,
SRM Institute of Science and Technology,
Ramapuram Campus, Chennai.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

RAMAPURAM, CHENNAI -600089

DECLARATION

We hereby declare that the entire work contained in this mini project report titled **"POISE: AN AI-BASED INTERVIEW PRACTICE AND FEEDBACK SYSTEM"** has been carried out by **KA DHARAN BALAJI [RA2411003020260], ABISHEK V.P.T [RA2411003020266], and V VARADHARAJAN [RA2411003020267]** at SRM Institute of Science and Technology, Ramapuram Campus, Chennai - 600089.

Place: Chennai

Date: 13.10.2026

ABSTRACT

Job interviews test more than technical knowledge: candidates must explain their thinking aloud, clearly and with evidence, under time pressure. Students rarely get to rehearse this realistically, because mock interviews with mentors are scarce and online question lists give no feedback on how an answer actually sounded. This project presents Poise, a full-stack web application for AI-based interview practice. It runs mock interviews with a spoken AI interviewer, generates questions for ten role tracks and four interview formats, tailors them to a pasted job description, captures answers through the browser's speech recognition and asks adaptive follow-up questions when an answer is vague. Every answer is scored from 0 to 100 on relevance, depth, structure, clarity and confidence; the scores are weighted into an overall score and a readiness band, and the feedback lists strengths, improvements, missed key points and a stronger sample answer. Each session ends with a debrief and a three-drill practice plan, and a dashboard tracks progress over time. Poise is built on the MERN stack in TypeScript, with a React 19 client, an Express 5 REST API and MongoDB. Its AI layer has two interchangeable engines: Claude, called through structured outputs, and an offline engine with a curated bank of 115 questions and a transparent rubric scorer, which also serves as the automatic fallback. All 40 automated tests pass. On the full question bank, the offline scorer ranked model, partial and vague answers in the correct order for every question, with mean scores of 84.5, 64.6 and 24.7, and typical API requests completed in about 12 ms.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	ABSTRACT	iv
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	x
	INTRODUCTION	1
	1.1 Overview	1
1	1.2 Problem Statement	2
	1.3 Objective of the Project	3
	1.4 Scope of the Project	4
	SOFTWARE AND HARDWARE SPECIFICATIONS	5
2	2.1 Hardware Requirements	5
	2.2 Software Requirements	5
3	**PROJECT DESCRIPTION**	8
	3.1 Introduction	8
	3.2 Architecture Diagram	9
	3.3 Algorithm Used	12
	3.4 Advantages of Algorithm Used	19
	3.5 Explanation of the Project	20

3.6 Output Results 26

3.7 Conclusion 33

APPENDIX 35

Source Code 35

LIST OF TABLES

Table No	Table Name	Page No.
2.1	Software stack and versions used	7
3.1	Scoring dimensions, weights and evidence	15
3.2	Score bands	16
3.3	REST API endpoints	22
3.4	Overall scores of the offline engine on the 115-question bank	30
3.5	API response times in milliseconds on a local deployment	32
3.6	Automated test results	33

LIST OF FIGURES

Fig No.	Figure Name	Page No.
3.1	Use case diagram of Poise	9
3.2	System architecture of Poise	10
3.3	Module structure of the Poise monorepo	11
3.4	Data model of Poise	12
3.5	Question generation and selection flow	13
3.6	Answer evaluation pipeline	17
3.7	Adaptive follow-up decision flow of the offline engine	18
3.8	Engine router with automatic fallback	19
3.9	Interview lifecycle sequence diagram	24
3.10	Continuous integration and deployment of Poise	25
3.11	Landing page of Poise	26
3.12	Interview setup screen	26
3.13	Mock interview room with live delivery metrics	27
3.14	Session debrief report	27

3.15	Per-question feedback with a highlighted filler word	28
3.16	Question Lab with instant feedback on a practice answer	28
3.17	Progress dashboard	29
3.18	(a) Interview history and (b) dashboard on a phone	29
3.19	Questions available to each role in the offline question bank	30
3.20	Distribution of overall scores by answer type	31
3.21	Mean dimension scores by answer type	31

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CI	Continuous Integration
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
HR	Human Resources
HTTPS	HyperText Transfer Protocol Secure
JD	Job Description
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MERN	MongoDB, Express, React and Node.js
ODM	Object Document Mapper
REST	Representational State Transfer
SPA	Single-Page Application
STAR	Situation, Task, Action, Result
STT	Speech-to-Text
TTS	Text-to-Speech
UI/UX	User Interface / User Experience
wpm	Words per minute

CHAPTER 1

INTRODUCTION

1.1 Overview

The interview is the decisive step between education and employment. In campus placements and entry-level hiring, candidates are asked to explain technical concepts, walk through their projects, design systems and tell stories about teamwork and failure, all within a few minutes per question. Knowing the material is necessary but not sufficient: interviewers also judge whether an answer addresses the question that was asked, whether it is supported by specific examples, whether it is structured so that it can be followed, and whether it is delivered with confidence rather than with a stream of filler words and hedges. These skills improve with deliberate practice and honest feedback, yet most students prepare by silently reading lists of questions and model answers.

Poise is a web application built to make that practice realistic and measurable. A candidate chooses a role from ten tracks, from Software Engineer to UI/UX Designer, an interview format (technical, behavioural and HR, mixed, or system design), a seniority level and a difficulty, and can paste a job description so that the questions follow the skills it mentions. The interview then takes place in a focused interview room where an animated interviewer reads each question aloud. The candidate answers by speaking or typing, sees live delivery metrics such as speaking pace and filler-word count while answering, and receives follow-up questions when an answer is vague or incomplete, much as a real panel would probe. When the interview ends, Poise evaluates every answer, writes a session debrief with a practice plan and adds the result to a progress dashboard.

Technically, Poise is a MERN application (MongoDB, Express, React and Node.js) written end to end in TypeScript and organised as a monorepo with three packages: the React client, the Express server and a shared package that holds the role catalogue, the scoring weights, the API types and the delivery-analysis code used by both sides. The artificial intelligence (AI) layer is pluggable. When an Anthropic API key is configured, Claude, a large language model (LLM), writes the questions, follow-ups, evaluations and debriefs through schema-validated structured outputs. Otherwise, and whenever a Claude call fails, a fully offline engine built on a curated question bank and a transparent scoring rubric takes over, so the application always works, even without internet access.

1.2 Problem Statement

Students preparing for interviews face four practical problems. First, realistic practice is hard to obtain: friends ask easy questions, mentors have limited time and paid coaching is expensive. Second, there is no objective feedback: candidates rarely notice their own filler words, hedging phrases, rambling sentences or the missing result at the end of a story, and self-assessment of content is unreliable. Third, preparation material is generic: question lists are not adapted to the target role, the experience level or a specific job description, and they never ask follow-up questions. Fourth, many online tools require sign-up, a constant internet connection and a paid AI service, and some upload audio or video recordings to remote servers, which raises privacy concerns.

The problem addressed by this project is therefore to design and implement a web application that can conduct a realistic, voice-driven mock interview for a chosen role and level, generate relevant and non-repetitive questions, evaluate each answer consistently and explainably on both content

and delivery, adapt to weak answers with follow-up questions and track improvement over time, while working without sign-up, without depending on the availability of an external AI service and without storing the candidate's audio or video.

1.3 Objective of the Project

The main objective of this project is to build Poise, a complete AI-based interview practice and feedback system. The specific objectives are:

- To develop a full-stack MERN application in TypeScript with a responsive React client, a secure Express REST API and a MongoDB database.
- To generate interview questions for ten roles, four formats, four experience levels and three difficulty levels, tailored to a job description, a company and chosen focus areas.
- To conduct spoken mock interviews in the browser, using speech synthesis for the interviewer's voice and speech recognition for the candidate's answers, with live delivery metrics.
- To evaluate every answer on five weighted dimensions and return specific, evidence-based feedback, missed key points and a stronger sample answer.
- To ask adaptive follow-up questions when an answer is too brief, lacks a result or a personal contribution, or skips trade-offs.
- To integrate a large language model through structured outputs while guaranteeing uninterrupted operation through an offline engine and automatic fallback.
- To track progress through a dashboard, an interview history and a personalised practice plan, while keeping each user's data private and secure.

1.4 Scope of the Project

Poise is intended for students and early-career professionals preparing for campus placements and entry-level interviews in software engineering, data science and analytics, DevOps, mobile development, product management and design. Within this scope the system covers the complete practice loop: setting up an interview, answering questions by voice or text, receiving follow-up questions, reviewing a detailed debrief and practising weak areas in the Question Lab. It also includes progress analytics, an interview history, profile settings and the option to erase all personal data.

The application runs in any modern browser on desktops, laptops and phones, with spoken answers supported in Chrome, Edge and Safari through the Web Speech API. It can be deployed as a single Docker container with MongoDB, on any Node.js host with MongoDB Atlas, or built as a single standalone HTML file that runs entirely in the browser for offline demonstrations. Audio and video are processed only in the browser; the server stores the text of answers, never recordings.

Coding rounds with an editor and test execution, résumé (PDF) analysis, server-side speech-to-text, analysis of facial expressions or body language, and peer or mentor review of reports are outside the scope of the present version and are identified as future enhancements.

CHAPTER 2

SOFTWARE AND HARDWARE SPECIFICATIONS

2.1 Hardware Requirements

The hardware requirements for developing and running this project are as follows:

• Processor: Minimum Intel Core i3 or AMD Ryzen 3 (dual core); Intel Core i5 or AMD Ryzen 5 recommended

• Memory (RAM): Minimum 8 GB; 16 GB recommended to run the Vite development server, the Node.js API and MongoDB together

• Hard disk: Minimum 10 GB of free space (SSD recommended) for Node.js, the project dependencies, MongoDB data and Docker images

• Network: Ethernet (LAN) or a wireless adapter (Wi-Fi); internet access is needed only to install dependencies and to use the optional Claude engine

• Microphone: Built-in or external microphone for spoken answers

• Webcam (optional): For the camera self-view used to practise eye contact

• Client device: Any modern desktop, laptop or smartphone; the interface adapts from 360-pixel-wide phone screens to wide desktop monitors

2.2 Software Requirements

The software requirements for this project are as follows:

• Node.js and npm - Node.js is a JavaScript runtime built on the V8 engine that executes JavaScript outside the browser. It runs the Express API and all the build tools of this project; version 20.19 or later is required and version 22 is recommended. npm workspaces manage the client, server and shared packages of the monorepo from a single root, so one command installs, tests and builds

the whole system.

• TypeScript - TypeScript is a typed superset of JavaScript. It is used in every package of Poise with strict type-checking. Because the client and the server import the same type definitions from the shared package, a change to the shape of an API response is caught at compile time on both sides.

• React and Vite - React is a component-based library for building user interfaces. Poise uses React 19 for its single-page client, with React Router for navigation, TanStack Query for fetching and caching server data, Tailwind CSS for styling, Motion for animations and Recharts for the dashboard charts. Vite provides the development server with hot module replacement and produces the optimised, code-split production bundle.

• Express - Express is a minimal and flexible web framework for Node.js. Poise uses Express 5 for its REST API, together with Helmet for security headers, CORS for cross-origin control, express-rate-limit for request throttling, Zod for request validation, jsonwebtoken for session tokens and bcrypt for hashing secrets.

• MongoDB and Mongoose - MongoDB is a document-oriented NoSQL database that stores data as flexible JSON-like documents. It suits Poise because an interview session, with its questions, answers, metrics and evaluations, is naturally a single document. Mongoose, an object document mapper (ODM), adds schemas, validation and indexes on top of the MongoDB driver.

• Anthropic SDK (Claude) - The official Anthropic SDK is used to call Claude, Anthropic's large language model, with structured outputs whose shape is defined by Zod schemas. This engine is optional and is enabled by setting an API key; the model is chosen through an environment variable.

• Web Speech API and MediaDevices - These browser APIs provide speech recognition (speech-to-text, STT), speech synthesis (text-to-speech, TTS), microphone access for the level meter and camera access for the self-view, without installing any additional software.

• Testing and DevOps tools - Vitest and Supertest are used for unit and API integration tests, tsup bundles the server, Docker and Docker Compose package the application with MongoDB, and Git with GitHub Actions provides version control and continuous integration (CI). Visual Studio Code was used as the code editor.

The exact versions of the main technologies used to build and test the project are listed in Table 2.1.

Table 2.1: Software stack and versions used

Layer	Technology	Version
Runtime	Node.js · npm	22.22 · 10.9
Language	TypeScript	5.9.3
Frontend library	React · React DOM	19.3.0
Build tool	Vite	7.3.7
Styling	Tailwind CSS	4.3.3
Server state	TanStack Query	5.104.2
Routing	React Router	7.18.4
Charts · animation	Recharts · Motion	3.10.1 · 12.43.0
Backend framework	Express	5.3.0
Validation	Zod	4.6.6
Database · ODM	MongoDB · Mongoose	7 (Docker image) · 8.24.5
AI SDK	Anthropic SDK	0.133.0
Security	jsonwebtoken · bcryptjs · Helmet	9.0.3 · 3.0.3 · 8.3.0
Testing	Vitest · Supertest	5.0.3 · 7.3.1
Packaging	tsup · Docker base image	8.5.1 · node:22-alpine

CHAPTER 3

PROJECT DESCRIPTION

3.1 Introduction

Poise follows the three-tier structure of a MERN application, with an additional AI layer. The presentation tier is a React single-page application (SPA) that also does real-time work in the browser: it converts speech to text, reads questions aloud, meters the microphone level, shows the camera preview and computes delivery metrics while the candidate speaks. The application tier is an Express API that identifies the user, validates every request, orchestrates interviews, evaluates answers in the background and aggregates analytics. The data tier is MongoDB, accessed through Mongoose. The AI layer sits behind a single interface with two implementations, a Claude engine and an offline engine, and a router that falls back from one to the other.

Figure 3.1 shows the main use cases. The candidate is the only human actor. There is no sign-up: the first visit creates a private browser profile automatically. From the dashboard the candidate can set up a mock interview, answer its questions by voice or text, receive adaptive follow-up questions, read the debrief and practice plan, drill single questions in the Question Lab, review past interviews and manage settings. The AI engine, either Claude or the offline engine, is a secondary actor that takes part in generating questions and follow-ups, evaluating answers and writing the debrief.

Five design principles guided the implementation. The same delivery metrics are computed in the browser and on the server by identical shared code, so the numbers a candidate sees while speaking match the report. The application must always work, so every AI call can fall back to the offline engine. Scores must be explainable, so each dimension is tied to observable evidence and the overall score is computed on the server with fixed weights for

both engines. Privacy is protected by keeping audio and video in the browser. Finally, the interview must feel responsive, so answers are evaluated in the background while the candidate moves straight on to the next question.

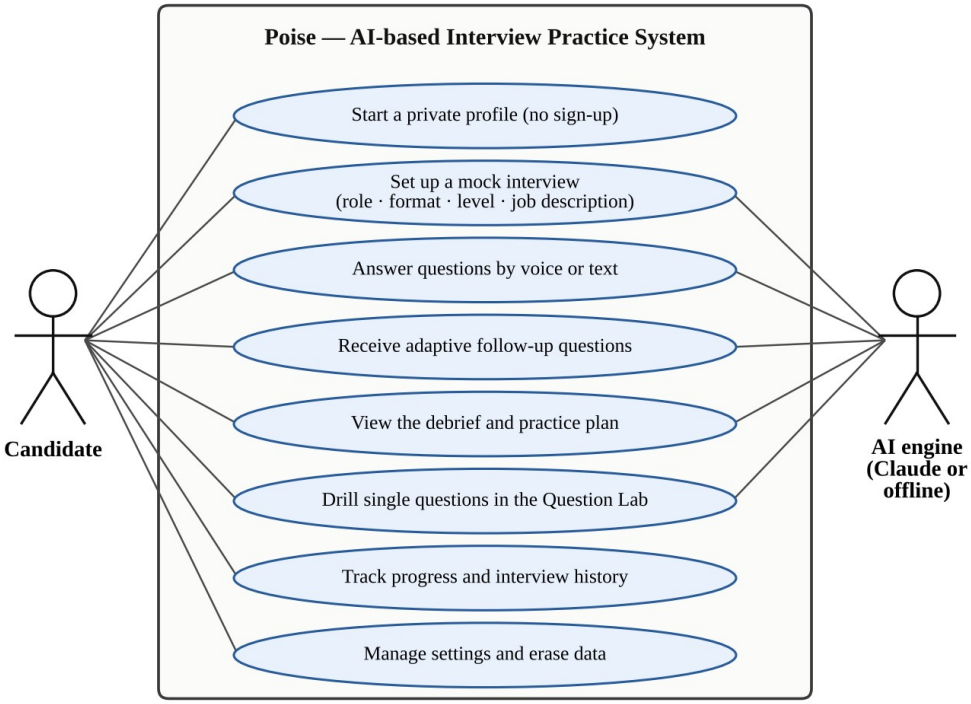


Figure 3.1: Use case diagram of Poise

3.2 Architecture Diagram

The architecture of Poise is shown in Figure 3.2. The browser hosts the React application, which talks to the server only through a REST API that exchanges JSON over HTTPS, with the session carried in an httpOnly cookie. The browser's Web Speech API turns the candidate's speech into text and reads questions aloud, the MediaDevices API provides the microphone level meter and the optional camera self-view, and the shared delivery-analysis code computes live metrics as the answer grows.

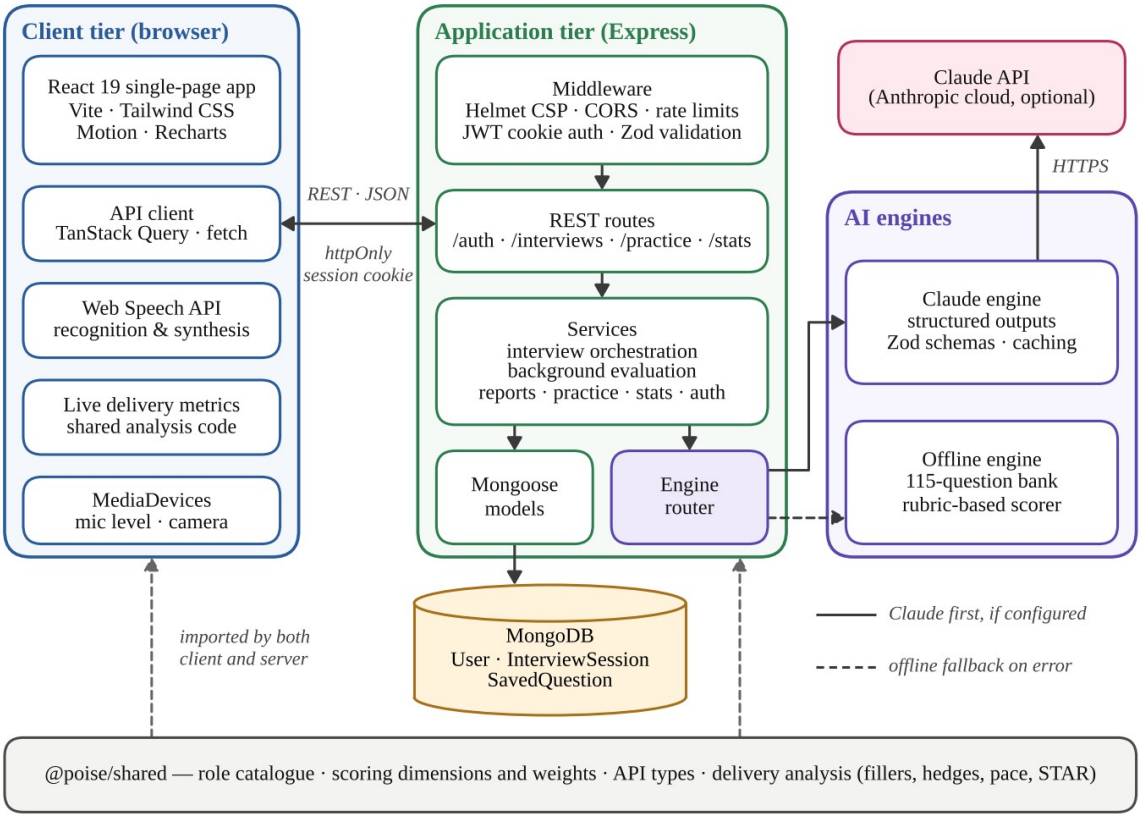


Figure 3.2: System architecture of Poise

On the server, every request passes through a middleware chain: Helmet sets security headers and a strict Content Security Policy (CSP), CORS restricts cross-origin access to the configured client origin, rate limiters protect the authentication and AI-backed endpoints, a JSON Web Token (JWT) check identifies the user from the session cookie and Zod schemas validate the request body. Routes delegate to services. The interview service creates sessions, records answers, schedules background evaluation and builds the final report; it reaches the AI engines only through the engine router and the database only through Mongoose models. The browser never calls Claude directly, so the API key stays on the server.

The code is organised as an npm workspaces monorepo, shown in Figure 3.3. The client package contains the pages, reusable components, hooks for speech and media, and the API client. The server package contains the routes

and middleware, the services, the AI engines, the Mongoose models and a core folder of pure functions with no database or network dependencies. Both packages import the shared package. The standalone build of the client reuses the server's core functions and offline engine, which is why it behaves exactly like the real API while running entirely in the browser.

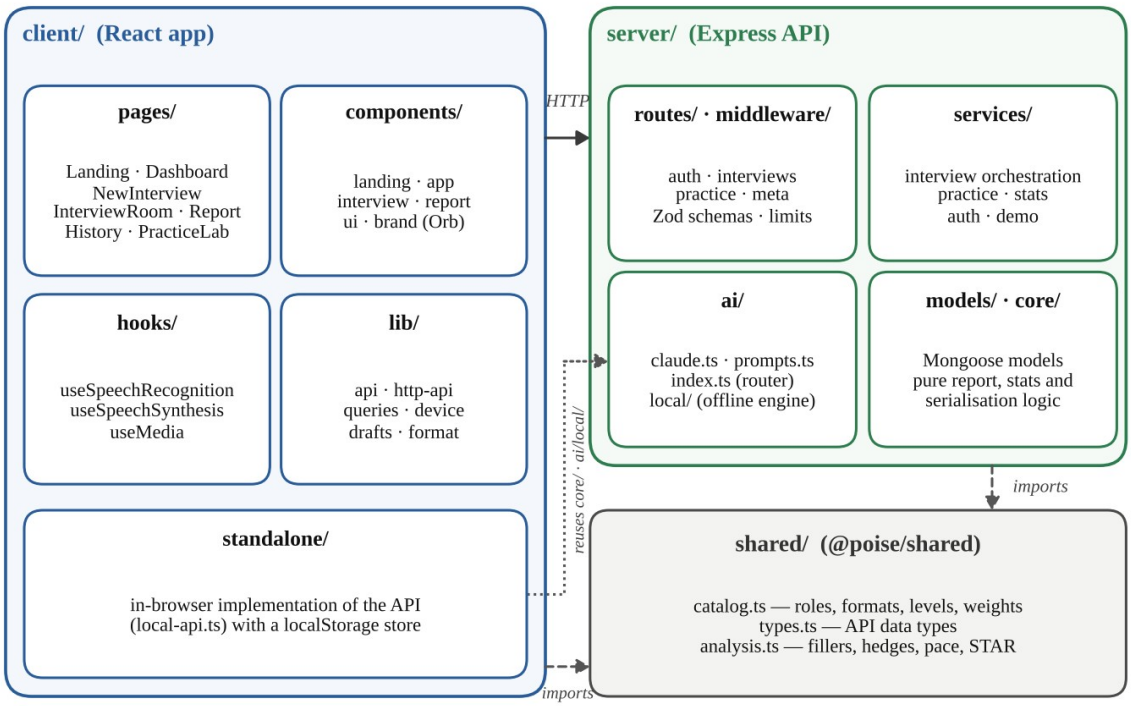


Figure 3.3: Module structure of the Poise monorepo

Figure 3.4 shows the data model. A user owns any number of interview sessions and saved questions. Questions are embedded inside their session rather than stored in a separate collection, because they are always read together with the session and a session is bounded to at most twelve core questions plus follow-ups. Each embedded question carries its answer, the computed delivery metrics and the evaluation, and the session carries the final report once it is complete. Saved questions form their own collection because they outlive any session; a unique index on the user and the question text prevents duplicates, and sessions are indexed by user and creation date for the dashboard and the

history. The reference answer of a question stays on the server and is never sent to the browser before the candidate has answered.

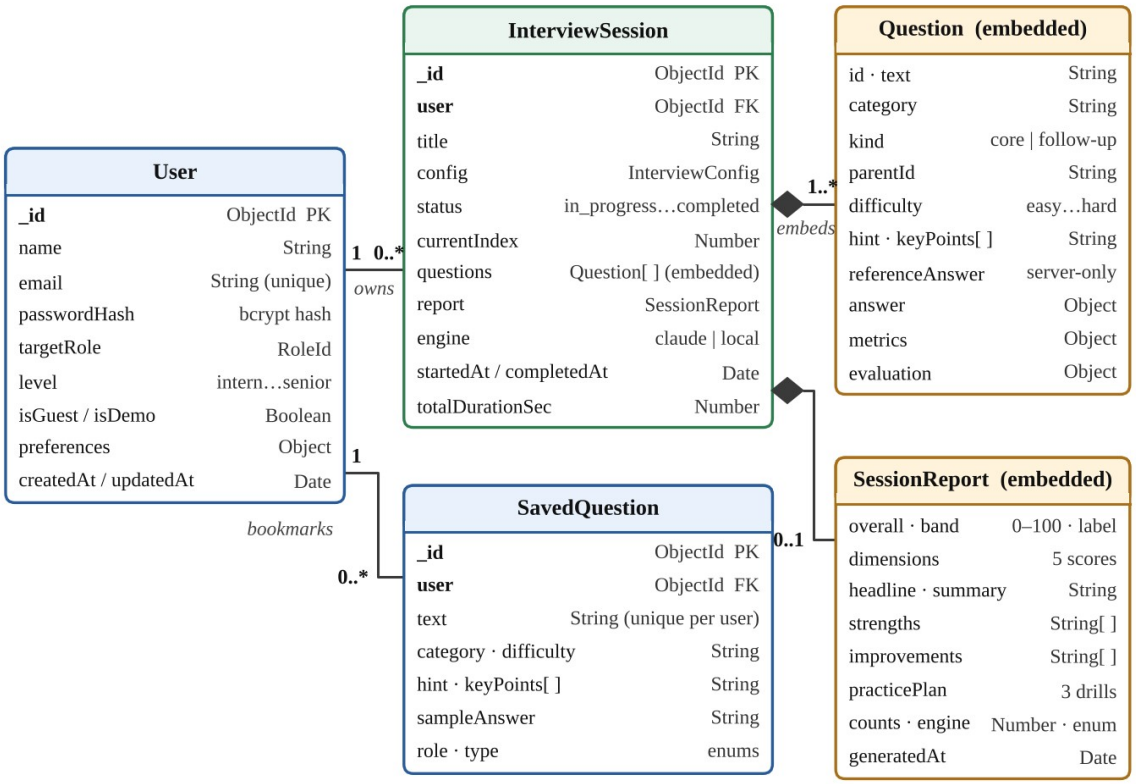


Figure 3.4: Data model of Poise

3.3 Algorithm Used

Poise combines five algorithms: question generation and selection, delivery analysis, rubric-based answer scoring, adaptive follow-up selection, and engine routing with automatic fallback. Question generation, scoring and follow-up selection are implemented twice, once as prompts and schemas for Claude and once as deterministic code in the offline engine, while the scoring weights, the score bands and the delivery analysis are shared by both engines.

3.3.1 Question Generation and Selection

Question generation starts from the interview configuration and the texts of the questions asked in the user's last five sessions, which are passed to the engine as questions to avoid. When Claude is configured, it receives the role,

level, format, difficulty, focus areas and job description (JD) and returns the requested number of questions as a schema-validated structured output, each with a category, a difficulty, a hint, key points and a reference answer. The offline engine selects questions from its bank of 115 questions using the procedure in Figure 3.5.

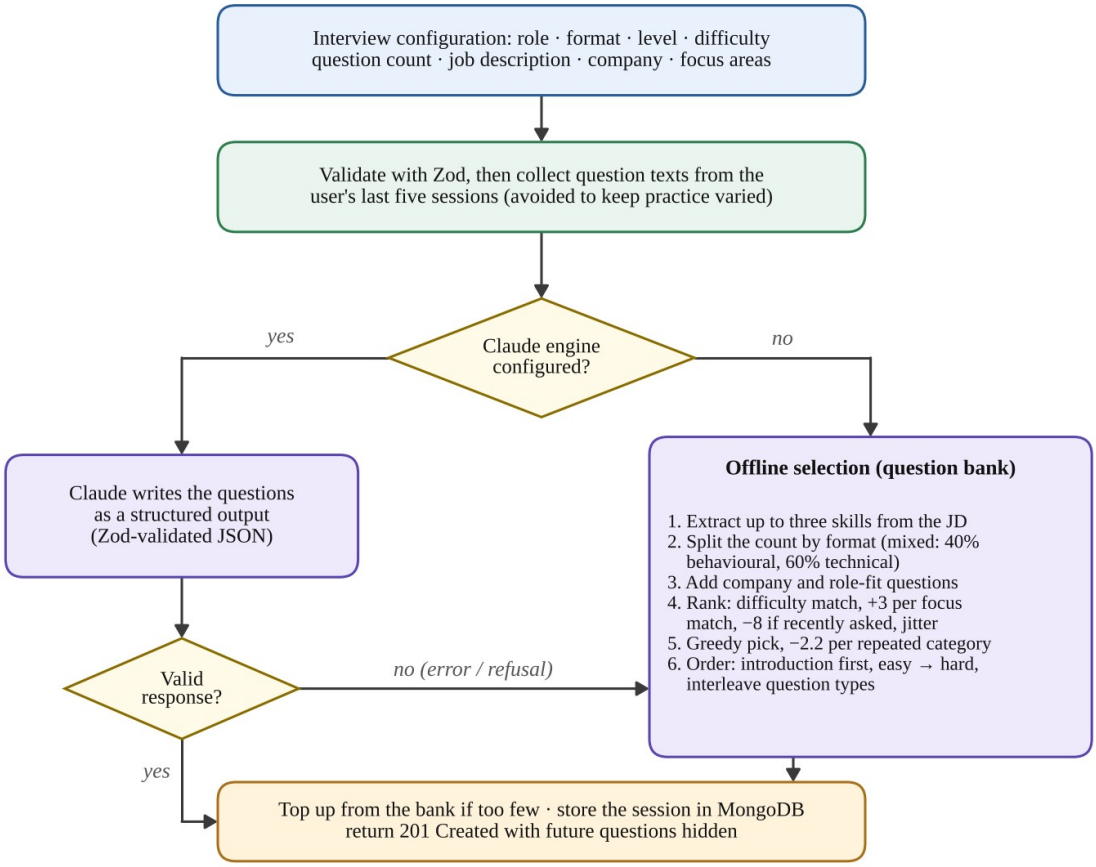


Figure 3.5: Question generation and selection flow

The offline engine first extracts up to three skills from the job description using a dictionary of skills and their aliases, and splits the requested count between question types: a mixed round is 40 per cent behavioural and 60 per cent technical, and a system-design round gives 60 per cent of its questions to design for engineering roles. A company name adds a motivation question and each extracted skill can add a role-fit question. Every remaining bank question q is then scored by Equation (1).

$$\text{score}(q) = 6 - 2.5\,\Delta(q) + 3\,F(q) - 8\,R(q) + \varepsilon \qquad (1)$$

Here $\Delta(q)$ is the distance between the question's difficulty and the requested difficulty (0, 1 or 2 steps), $F(q)$ is the number of focus areas or job-description skills found in the question, $R(q)$ is 1 if the question was asked in the last five sessions and 0 otherwise, and ε is random jitter drawn uniformly between 0 and 2.5 so that repeated sessions differ. Questions are picked greedily by score, and every question already chosen from a category lowers the effective score of the remaining questions in that category by 2.2, which keeps a round varied. Finally, the round is ordered like a real interview: “Tell me about yourself” comes first in behavioural and mixed rounds, easier questions come before harder ones and question types are interleaved.

3.3.2 Delivery Analysis

The delivery-analysis module computes measurable properties of an answer and is shared by the browser and the server. The text is tokenised and each token is classified as a filler, a hedge or neither by matching a list of seventeen filler phrases, such as “um”, “basically”, “you know” and “I mean”, and twelve hedging phrases, such as “I think”, “maybe” and “kind of”; longer phrases are matched first so that “I feel like” counts once. The word “like” is ambiguous, so it is counted as a filler only in filler positions, for example after a comma or after “and”, “so” or “was”, and never before words such as “to”, “this” or “a”. For spoken answers of at least ten seconds, the speaking pace in words per minute (wpm) is computed by Equation (2).

$$\text{wpm} = W \div (T \div 60) \qquad (2)$$

where W is the number of words and T is the duration of the answer in seconds. Speech-to-text transcripts often lack punctuation, so when an answer of 40 or more words appears to have fewer than one sentence per 45 words, the

sentence count is estimated from a typical spoken sentence length of 16 words instead of penalising clarity for a transcription artefact. The module also detects the four parts of the STAR method (situation, task, action and result) from phrase markers and from the pattern “I” followed by a past-tense verb, and records whether the answer contains numbers or examples.

3.3.3 Rubric-based Answer Scoring

Every answer receives an integer score from 0 to 100 on five dimensions. Table 3.1 lists each dimension, its weight in the overall score and the evidence that the offline engine uses to score it.

Table 3.1: Scoring dimensions, weights and evidence

Dimension	Weight	Question it answers	Evidence used by the offline engine
Relevance	25%	Did the answer address the question that was asked?	Key-point coverage and overlap with the question's content words; capped for very short answers
Depth	25%	Was it accurate, specific and backed by examples?	Length curve peaking at 110–260 words, key-point coverage, numbers (+8), examples (+8), trade-offs (+6)
Structure	20%	Could the interviewer follow it?	STAR parts for stories; signposting connectors, sentence count and a direct opening for explanations
Clarity	15%	Was the language concise and easy to follow?	Starts at 92; penalties for long sentences and for filler density
Confidence	15%	Did the candidate sound decisive?	Starts at 86; penalties for hedges, fillers and a pace outside 100–175 wpm; bonus for assertive phrasing

Key points are matched through explicit match terms written for each bank question: short terms such as “TTL” must match whole words, while longer terms match as prefixes, so that “cach” matches both “cache” and “caching”. For questions without explicit terms, a light stemmer derives the terms from the key point itself. The coverage c is the share of key points

matched and the overlap o is the share of the question's content words echoed in the answer. Relevance, for example, is computed by Equation (3).

$$\text{Relevance} = 20 + 55c + 25 \times \min(1, o \div 0.45) \quad (3)$$

An answer that covers no key point and barely overlaps the question is treated as off-topic, and its structure, clarity and confidence are capped so that fluent delivery cannot rescue an irrelevant answer. The overall score S is then computed on the server, for both engines, from the five dimension scores by Equation (4) and mapped to the score bands in Table 3.2.

$$S = \text{round}(0.25\, s_{rel} + 0.25\, s_{dep} + 0.20\, s_{str} + 0.15\, s_{cla} + 0.15\, s_{con}) \quad (4)$$

Table 3.2: Score bands

Overall score	Band
85 – 100	Interview-ready
70 – 84	Strong
55 – 69	Promising
40 – 54	Needs practice
0 – 39	Early days

When Claude is the active engine, the same rubric is written into a stable system prompt with calibration anchors for each dimension (90 and above, around 60, and below 30) and expectations adjusted to the candidate's level; the prompt is marked for prompt caching because it is identical for every request. The candidate's answer is wrapped in <candidate_answer> tags and the model is instructed to treat it as data, never as instructions, which guards against prompt injection such as an answer that asks for full marks. The delivery metrics are passed as evidence for the clarity and confidence scores. The response must match a Zod schema; the server clamps every score to the range 0 to 100, trims the lists and computes the overall score itself. Figure 3.6 summarises the complete evaluation pipeline.

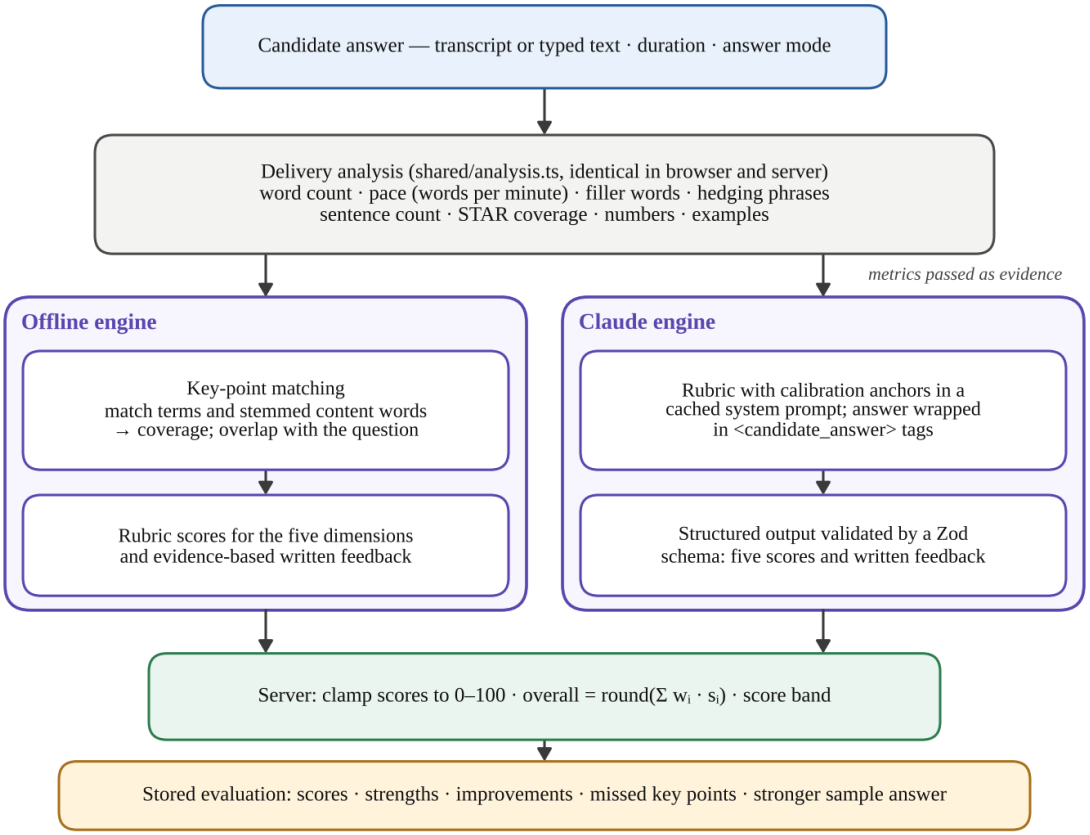


Figure 3.6: Answer evaluation pipeline

The offline engine writes its feedback from the strongest signals, for example “You used 3 filler words (‘um’ ×1, ‘basically’ ×1, ‘actually’ ×1)”, lists the key points covered and missed, and returns the bank's reference answer as the stronger sample answer. At the end of a session, the report builder averages the dimension scores of the answered questions, computes delivery habits such as fillers per 100 words and the average pace, and writes a headline, a summary, strengths, improvements and a three-drill practice plan aimed at the two weakest dimensions and the lowest-scoring answer.

3.3.4 Adaptive Follow-up Questions

Real interviewers probe weak answers, and Poise does the same. After each core answer, if adaptive follow-ups are enabled and the number of follow-ups is still below half the number of core questions, the engine decides whether to ask one. The offline engine applies the ordered rules in Figure 3.7 and targets

the biggest gap: a brief answer is asked for a concrete example, a behavioural story without a result is asked for the outcome, a story told only as “we” is asked for the candidate's own actions, and a technical answer without trade-offs or examples is asked for them, while a strong technical answer may be asked how it would be verified in production. Claude makes the same decision at low reasoning effort, because the candidate is waiting in the room. The chosen follow-up is inserted directly after the current question with a positional $push, so it becomes the next question.

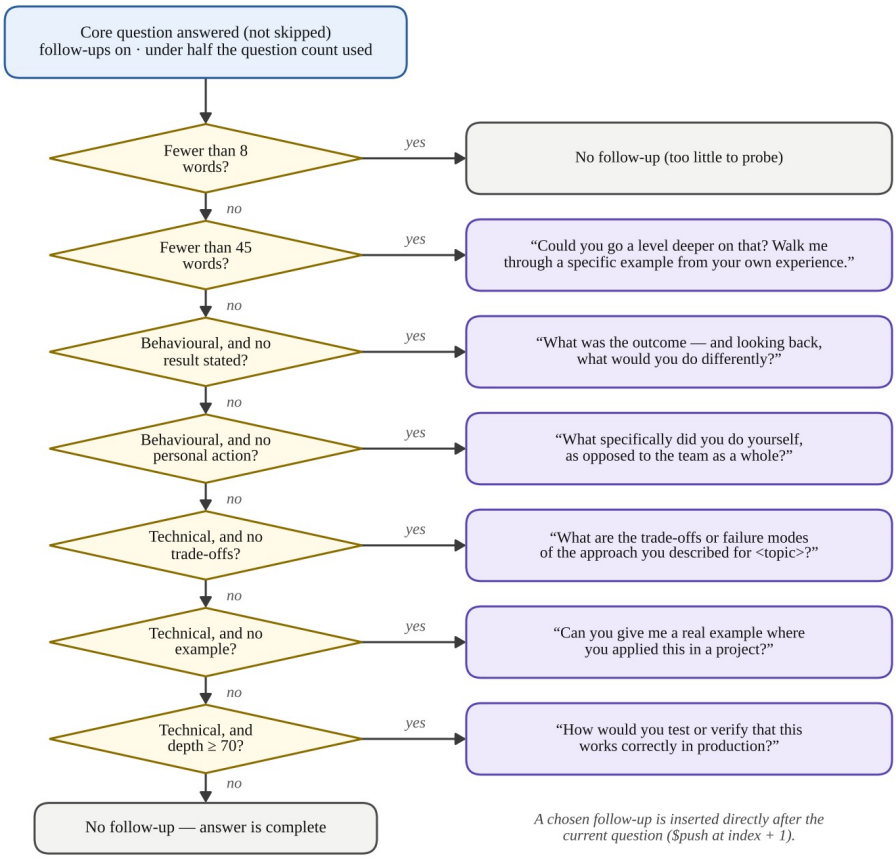


Figure 3.7: Adaptive follow-up decision flow of the offline engine

3.3.5 Engine Routing and Automatic Fallback

Every engine operation passes through a single function, runEngine, shown in Figure 3.8. In the default automatic mode, Claude is used when an API key is configured, and the AI_PROVIDER setting can force the offline engine or always try Claude. A Claude call uses structured outputs at medium

effort (low effort with a 45-second timeout for follow-ups), a 120-second client timeout and two automatic retries, and a server-side fallback model handles safety refusals. If the call still fails for any reason, such as an authentication error, a rate limit, a timeout, a refusal, a truncated response or output that does not match the schema, the router logs the reason and transparently runs the same operation on the offline engine. The name of the engine that actually produced each result is stored with it and shown in the interface.

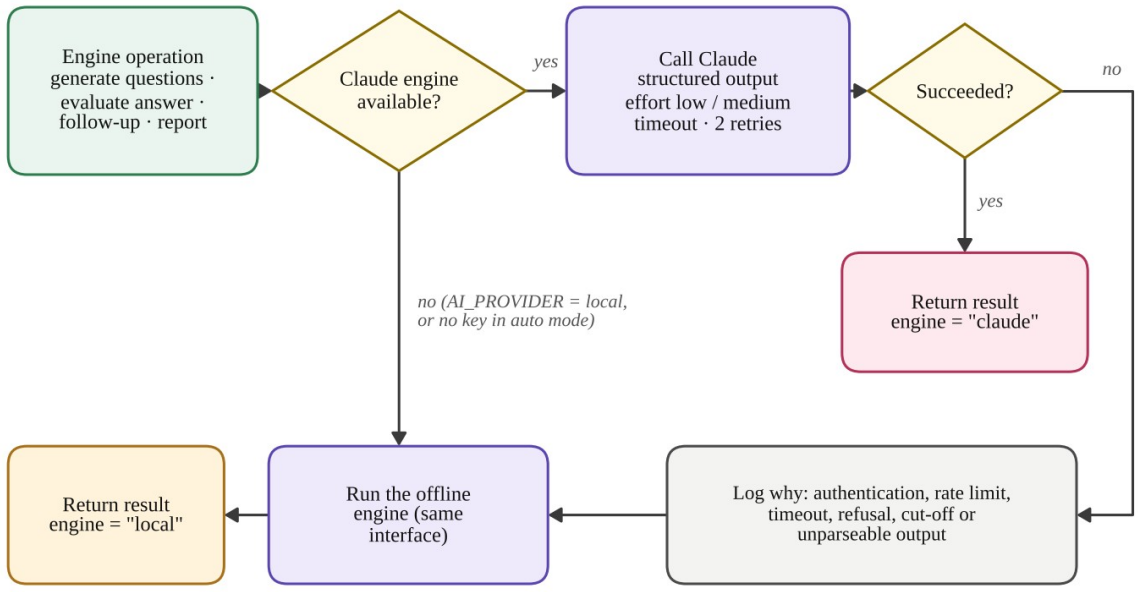


Figure 3.8: Engine router with automatic fallback

3.4 Advantages of Algorithm Used

- Consistency and explainability: the offline rubric is deterministic, so the same answer always receives the same score, and every score is tied to evidence that the feedback can cite, such as a missed key point or a filler-word count.
- Identical metrics everywhere: delivery analysis runs from the same shared code in the browser and on the server, so the live metrics shown during the interview match the stored report.

- Reliability: the offline engine and automatic fallback mean that an interview never stalls because of a missing key, a network failure or a rate limit, and the application works fully offline.
- Fairness across engines: the overall score and band are always computed on the server from the five dimension scores with the same weights, and Claude's scores are clamped, so results from both engines are comparable.
- Robustness to speech recognition: sentence estimation for unpunctuated transcripts and context rules for the word “like” stop transcription artefacts from lowering clarity or confidence.
- Security against prompt injection: the candidate's answer is passed to Claude as tagged data, so instructions hidden inside an answer cannot change its score.
- Efficiency and responsiveness: offline evaluation takes well under a millisecond, evaluation runs in the background while the candidate continues, and prompt caching and low effort reduce Claude's latency where the candidate is waiting.
- Realism and variety: recency penalties, category diversity, interview-like ordering and adaptive follow-ups make each session feel like a real interview rather than a quiz.

Overall, combining a large language model with a transparent rule-based engine gives Poise the fluency of AI-written questions and feedback when the model is available, together with the predictability, speed and offline availability of an algorithm whose every score can be explained.

3.5 Explanation of the Project

This section explains how the modules of Poise work together, from the user interface and the API to security, the interview lifecycle and deployment.

3.5.1 Client Application

The React client has eight main pages. The landing page introduces the product. The dashboard shows the average and best scores, a score trend over time, a skill profile averaged over the last ten interviews, delivery habits compared with targets, the practice streak, the roles practised and a “focus next” recommendation that launches a targeted session; an empty profile can be filled with a week of sample history to explore it. The setup page collects the role, format, level, difficulty, number of questions (3 to 12), answer mode, time per question (1 to 10 minutes, or untimed), adaptive follow-ups and the optional job description, company and focus areas, and estimates the length of the session.

The interview room is a dark, distraction-free stage. An animated orb represents the interviewer and visibly idles, speaks, listens to the microphone level and thinks. Each question is read aloud by speech synthesis, and the answer is captured by continuous speech recognition that restarts itself after pauses; the transcript stays editable before it is submitted. Word count, pace, filler words and hedges update live, a countdown timer shows the time left, hints are available on demand, drafts are saved locally so that a page refresh never loses an answer, and Ctrl+Enter submits. The report page shows the debrief, the five-dimension breakdown, delivery habits, the practice plan and a review of every question with the answer annotated for fillers and hedges, the key points covered and missed, and a stronger sample answer; it can be saved as a PDF. The history page lists all interviews with role and status filters, and the Question Lab generates questions on demand, gives instant feedback on single answers and lets the candidate bookmark questions into a personal bank.

3.5.2 REST API

The server exposes the REST API summarised in Table 3.3. All endpoints are under /api and exchange JSON. Every error uses one shape,

{ error: { code, message, details } }, so the client can show precise messages, and every interview query is scoped to its owner, so another user's interview returns 404 Not Found. Endpoints for registration, login, a shared demo account and password changes also remain available in the API, although the web application currently uses browser profiles.

Table 3.3: REST API endpoints

Method	Endpoint	Purpose
POST	/api/auth/guest	Start or restore a browser profile
GET	/api/auth/session	Current user; renews an active session
GET · PATCH · DELETE	/api/auth/me	Read, update or erase the profile
GET	/api/interviews	List interviews by status and role
POST	/api/interviews	Create an interview and its questions
POST	/api/interviews/sample	Add sample history to an empty profile
GET	/api/interviews/:id	Get an interview (polled while scoring)
POST	/api/interviews/:id/answers	Submit or skip the current question
POST	/api/interviews/:id/complete	Finish and build the debrief
DELETE	/api/interviews/:id	Delete an interview
POST	/api/practice/questions	Generate practice questions
POST	/api/practice/evaluate	Instant feedback on a single answer
GET · POST · DELETE	/api/practice/saved[/:id]	List, save or remove saved questions
GET	/api/stats	Dashboard analytics (user's time zone)
GET	/api/engine · /api/health	Active AI engine and health check

3.5.3 Security and Privacy

Security and privacy were treated as requirements from the start:

- No sign-up: the first visit creates a profile protected by a random 256-bit device key, which the browser keeps in local storage and the server stores only as a bcrypt hash; presenting the key restores the same profile even after the session cookie expires.

- Sessions are JSON Web Tokens in httpOnly, SameSite=Lax cookies that scripts cannot read, marked Secure in production and valid for seven days. Account passwords are hashed with bcrypt at 12 rounds, and login compares against a dummy hash for unknown emails so that response times do not reveal which emails exist.
- Every request body is validated with Zod, JSON bodies are limited to 200 kB, and Helmet applies security headers with a strict Content Security Policy. The profile endpoint accepts JSON only, so a form on another website cannot swap a visitor's profile.
- Rate limits allow 300 session starts and 30 password attempts per 15 minutes from one IP address, and 60 AI-backed calls per 10 minutes for each user.
- Audio and video never leave the browser; the server stores only the text of answers, and the user can erase the profile and all its data from the settings page.

3.5.4 Interview Lifecycle

Figure 3.9 shows the life of an interview. Creating an interview validates the configuration, collects the recently asked questions, generates the questions through the engine router, tops them up from the bank if the engine returned too few, and stores the session with the status in_progress. The response hides the text of the questions the candidate has not reached and the key points of the questions not yet answered, just as in a real interview.

When an answer is submitted, the server checks that it belongs to the current question, computes the delivery metrics and records the answer with a single atomic update conditioned on the status and the current index, so a double submission cannot record two answers. Evaluation is scheduled in the background and the response returns at once with the next question; if a follow-

up is chosen, it is inserted right after the current question. Evaluations and follow-ups write to different array elements with targeted positional operators, so concurrent updates never overwrite one another. Finishing the interview moves it to the evaluating state. When the last evaluation lands, the report builder aggregates the scores, writes the debrief and marks the session completed, while the client polls every 1.5 seconds and shows the progress of each answer. Unfinished work is queued again when the server restarts, so no interview is left half-evaluated.

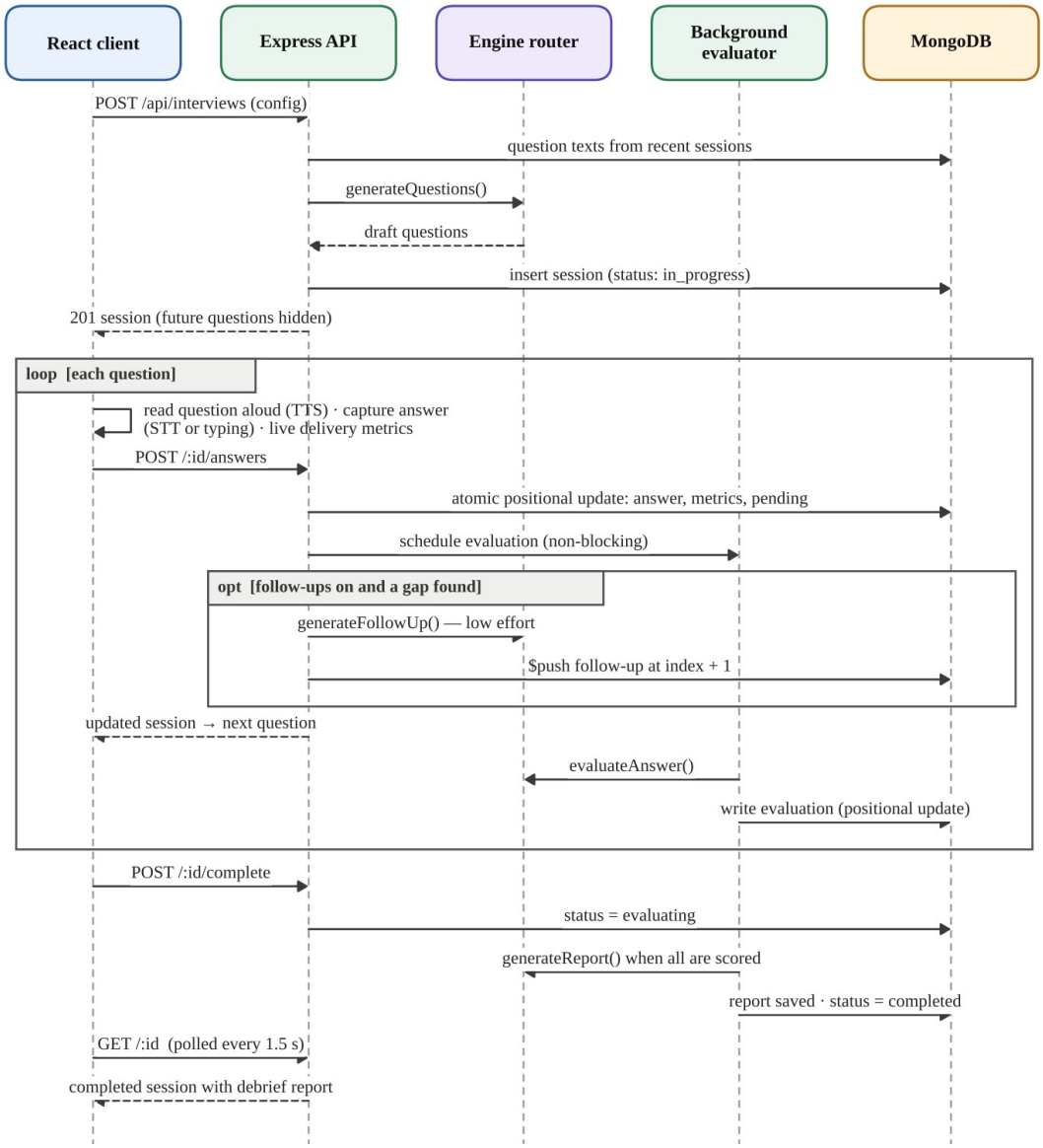


Figure 3.9: Interview lifecycle sequence diagram

3.5.5 Testing and Deployment

Automated tests cover the three layers: the shared delivery analysis, the offline engine (bank integrity, selection, tailoring, scoring order, feedback and follow-ups) and the whole API against a real MongoDB database, including the interview lifecycle, ordering rules, ownership isolation and profile restoration. The GitHub Actions workflow in Figure 3.10 runs the type-checks, the tests against a MongoDB service, the production build and the standalone build on every push and pull request. For deployment, a multi-stage Dockerfile compiles the client and the server on node:22-alpine and ships a slim runtime image in which one Node.js process serves both the API and the built React application on port 4000, and Docker Compose adds MongoDB with a persistent volume. The same build also runs on any Node.js host with MongoDB Atlas. HTTPS is required in production, because session cookies are marked Secure and browsers allow microphone access only on secure origins.

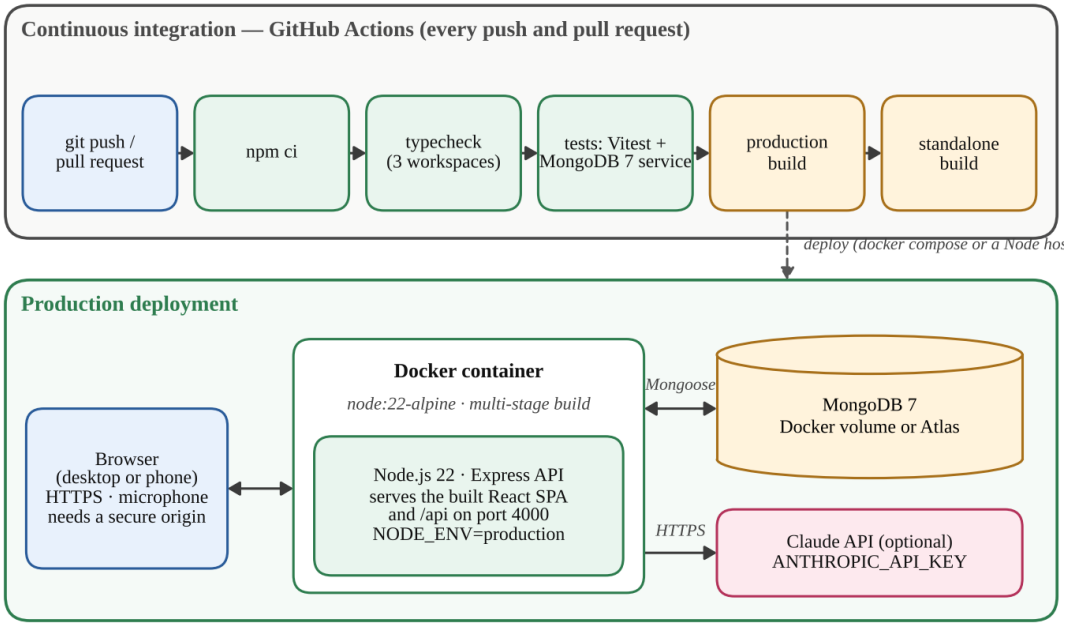


Figure 3.10: Continuous integration and deployment of Poise

3.6 Output Results

The outputs below were captured from the running application, built with npm run build and served by the Express server with the offline engine and a local MongoDB database. The dashboard, history and report screens use the sample history that the application can load into an empty profile. Figure 3.11 shows the landing page, which explains the product and starts a private profile without sign-up. The interview setup screen in Figure 3.12 lets the candidate choose one of the ten roles, the format, the level and the difficulty, and summarises the session with its estimated duration.

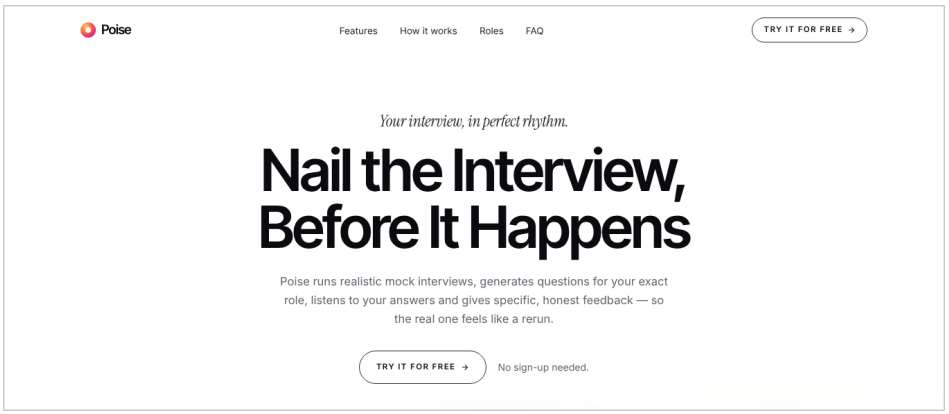


Figure 3.11: Landing page of Poise

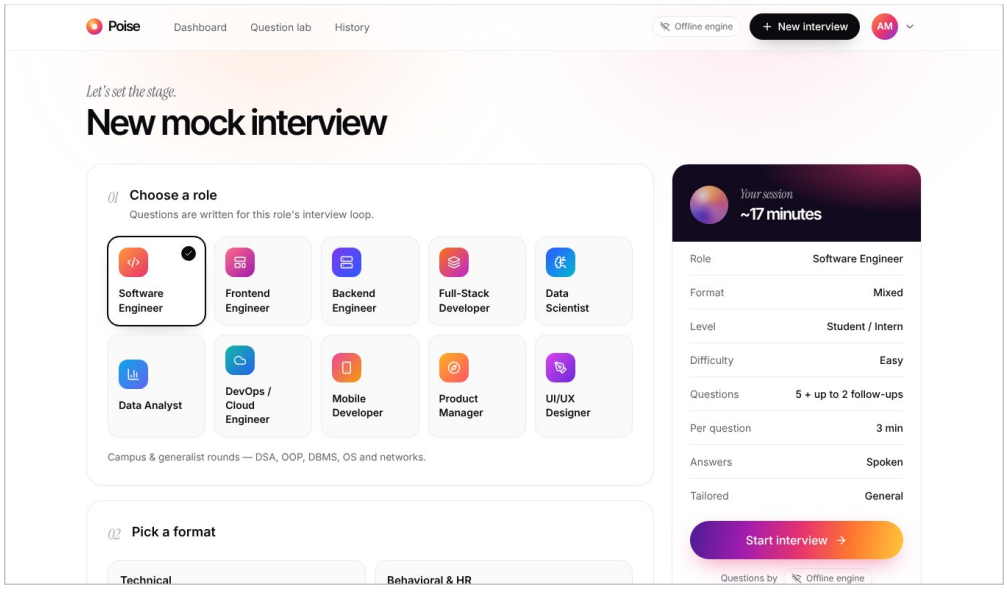


Figure 3.12: Interview setup screen

Figure 3.13 shows the interview room during a spoken answer. The question has been read aloud, the transcript appears as the candidate speaks, and the live metrics below it already count 43 words and two filler words (“um” and “like”). The debrief that follows the interview is shown in Figure 3.14.

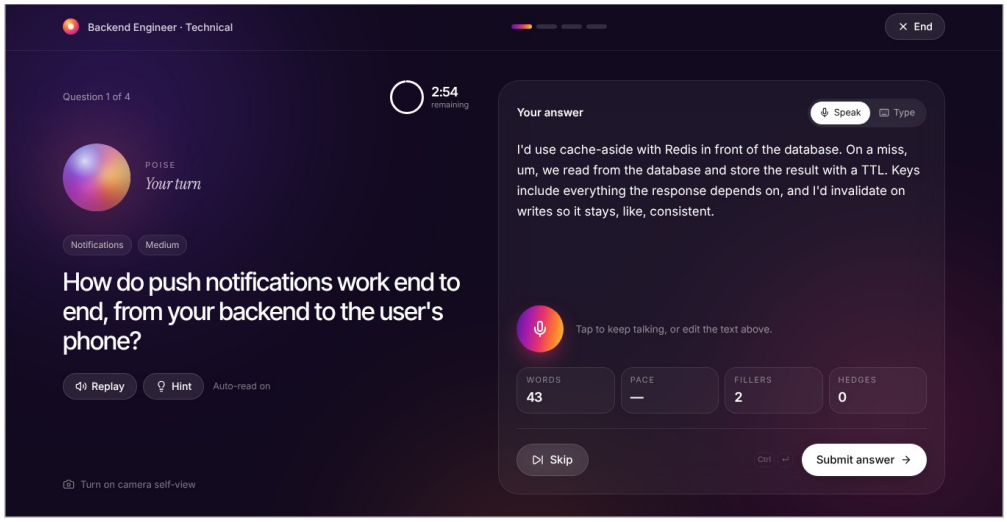


Figure 3.13: Mock interview room with live delivery metrics

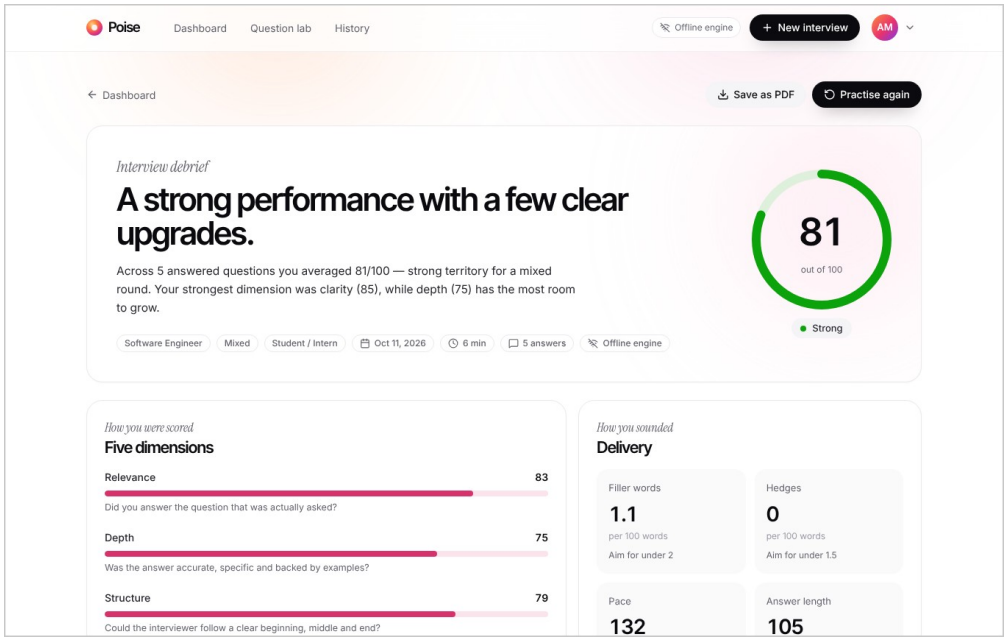


Figure 3.14: Session debrief report

The debrief summarises the session with an overall score of 81 in the Strong band, the five-dimension breakdown and delivery habits such as fillers and hedges per 100 words, pace and answer length. Each question can be

expanded as in Figure 3.15, which shows the candidate's answer with the filler word highlighted, the dimension scores, what worked and what to improve, and the key points that were covered.

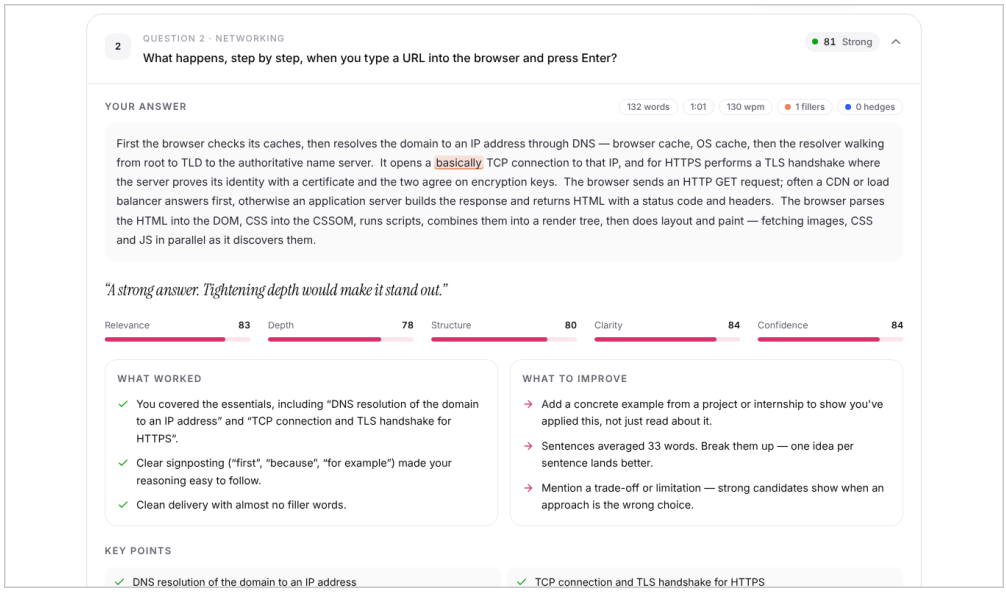


Figure 3.15: Per-question feedback with a highlighted filler word

The Question Lab in Figure 3.16 gives instant feedback on a single practice answer. For the answer shown, the engine highlighted three fillers and scored the answer 81 out of 100. It noted that the answer covered the B-tree structure of an index and the write and storage costs, and pointed out the missing idea that an index turns a full scan into a logarithmic lookup.

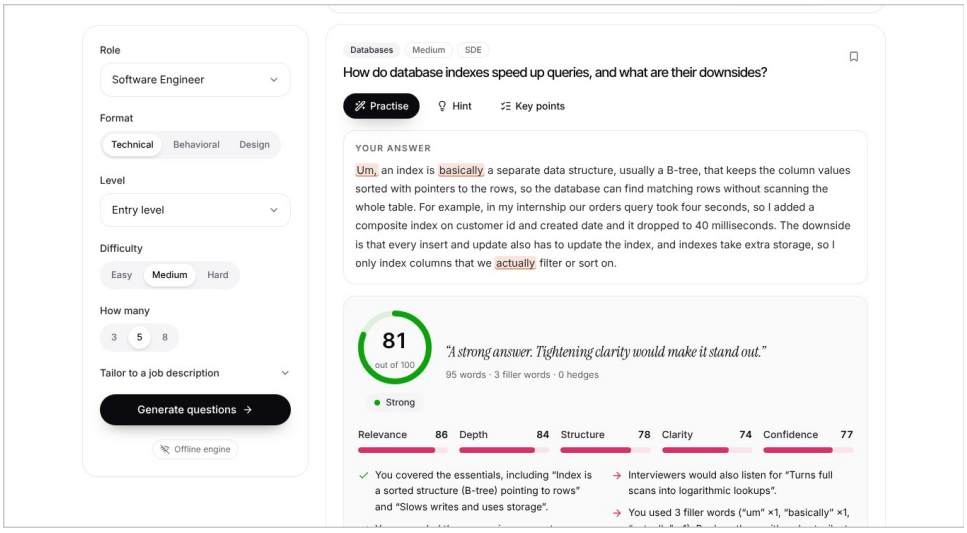


Figure 3.16: Question Lab with instant feedback on a practice answer

Figure 3.17 shows the progress dashboard with summary statistics, the score trend and the skill profile across the five dimensions, and Figure 3.18 shows the interview history together with the responsive layout on a phone screen 390 pixels wide.

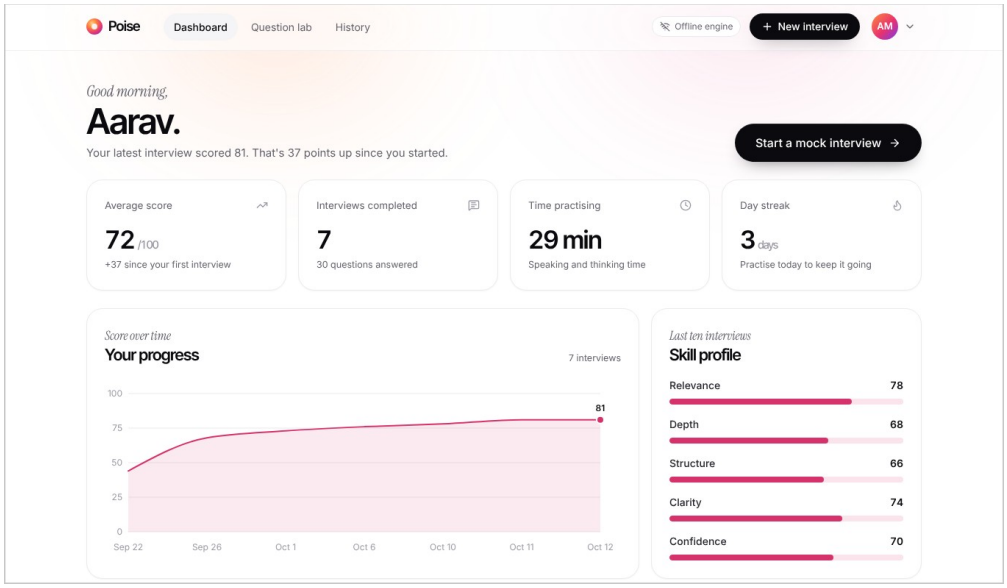
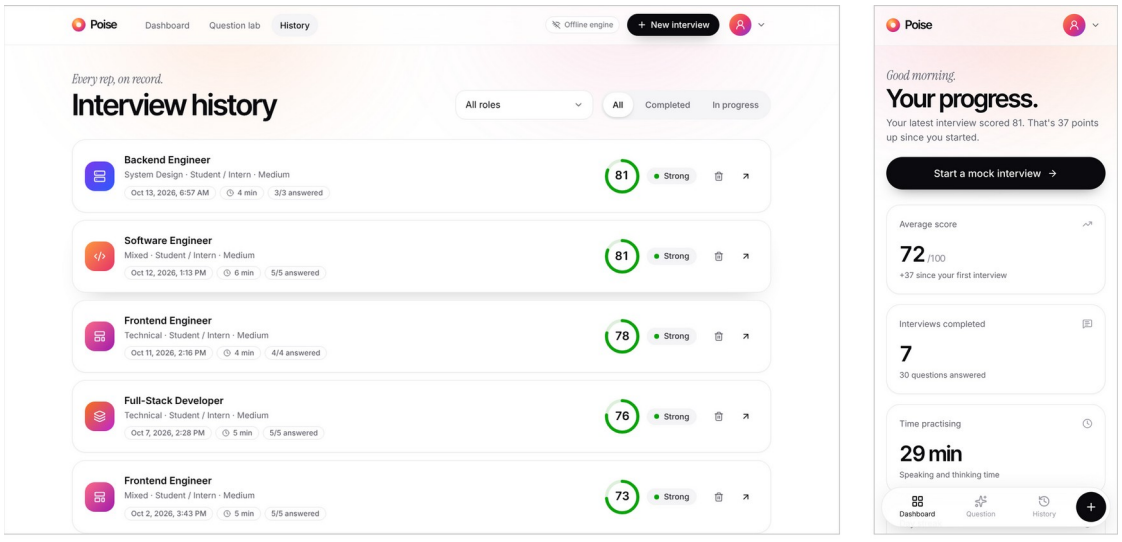


Figure 3.17: Progress dashboard



(a) (b)

Figure 3.18: (a) Interview history and (b) dashboard on a phone

To evaluate the offline engine beyond the unit tests, its question bank was analysed and its scorer was run on every one of the 115 bank questions. Figure 3.19 shows how many questions are available to each role: between 31

and 56, combining role-specific technical questions, behavioural questions shared by all roles and, for engineering roles, system-design questions. Across the bank there are 85 technical, 20 behavioural and 10 system-design questions in 72 categories, with 30 easy, 59 medium and 26 hard questions, an average of 4.1 key points per question and reference answers of about 120 words.

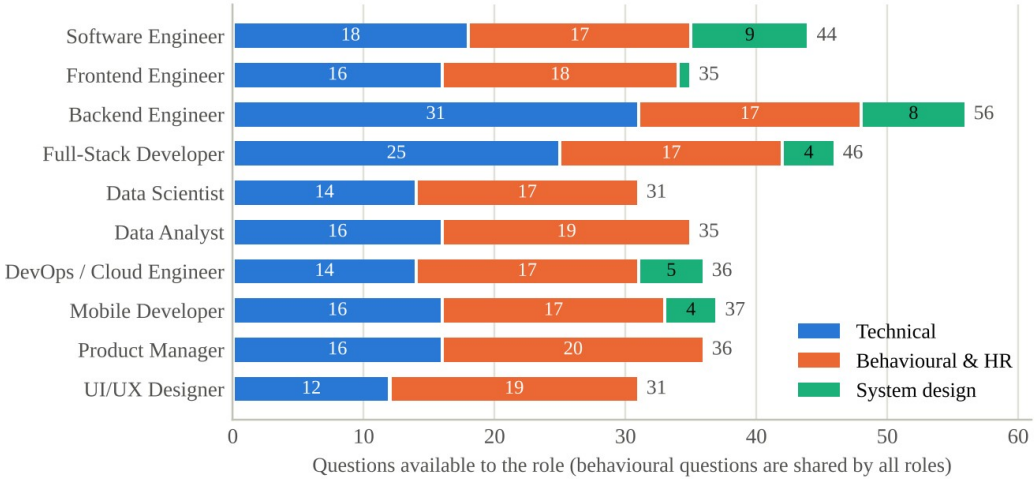


Figure 3.19: Questions available to each role in the offline question bank

For each question, four spoken answers were scored: the bank's reference answer (model), its first two sentences (partial), a fixed answer full of fillers and hedges (vague) and a fixed off-topic answer about hobbies. Table 3.4 and Figure 3.20 show that the engine separated them clearly: the order model > partial > vague held for all 115 questions, 98 of the 115 model answers (85 per cent) scored above 80, and 114 of the 115 off-topic answers (99 per cent) scored below 35.

Table 3.4: Overall scores of the offline engine on the 115-question bank

Answer type	Mean	Std. dev.	Min	Max	Time / answer
Model answer	84.5	4.8	67	96	0.29 ms
Partial answer	64.6	9.2	26	84	0.17 ms
Vague answer	24.7	2.5	23	36	0.16 ms
Off-topic answer	27.0	1.7	26	45	0.13 ms

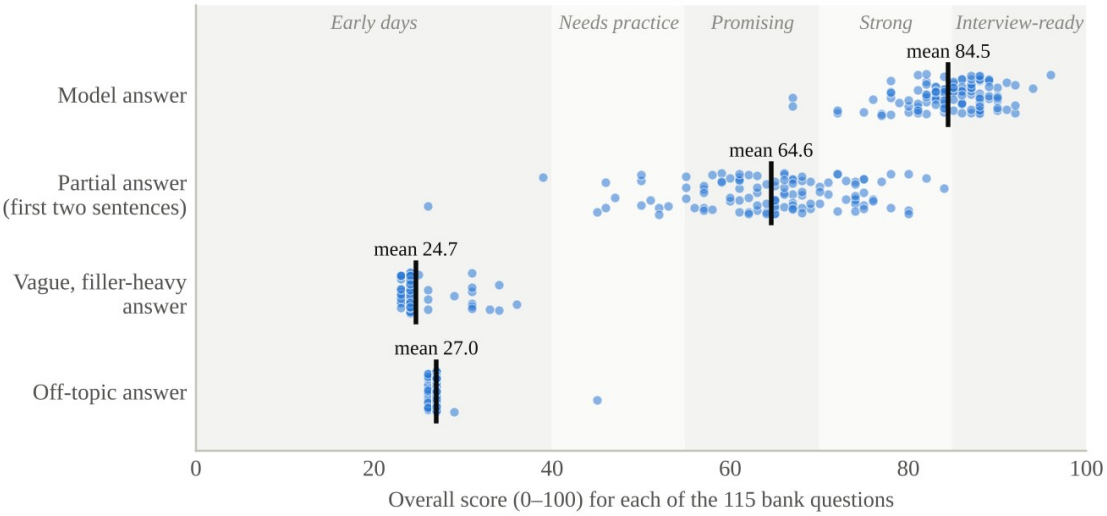


Figure 3.20: Distribution of overall scores by answer type

Figure 3.21 breaks the scores down by dimension. Relevance and depth fall sharply from model to partial answers, because the partial answer covers fewer key points and gives fewer specifics, while clarity and confidence stay high for both, because a shortened answer is still delivered clearly. The vague answer loses on every dimension, and its clarity and confidence are pulled down by fillers and hedges. This is the intended behaviour: the content dimensions measure what was said and the delivery dimensions measure how it was said.

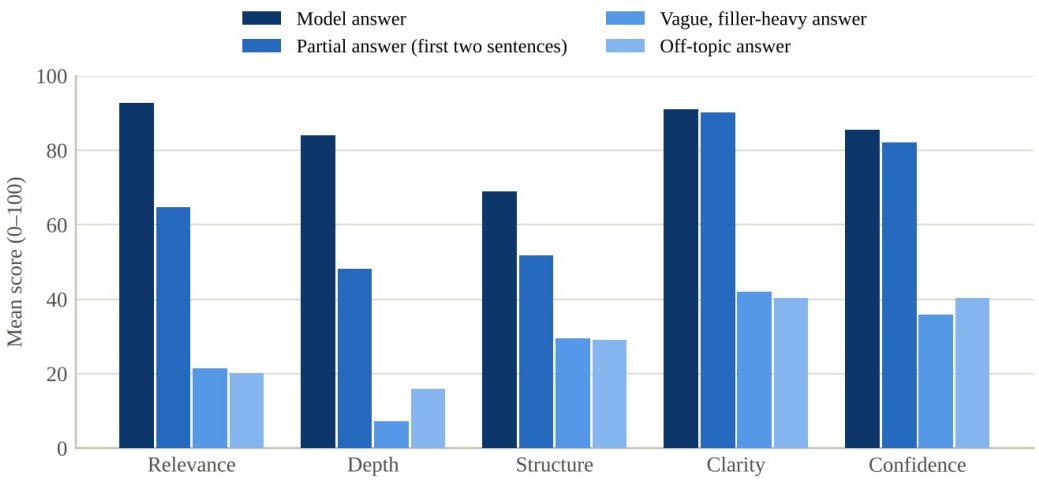


Figure 3.21: Mean dimension scores by answer type

The delivery analysis was also checked on a sample spoken answer of 69 words given in 32 seconds. It reported a pace of 129 words per minute, two fillers (“um” and “like”), one hedge (“I think”), four sentences, all four STAR components and the presence of numbers, which matches a manual reading of the answer.

The response times of the API were measured with the server code, the offline engine and a local MongoDB 8.3 database on a four-core 2.1 GHz Intel Xeon machine with 15 GB of memory, by running 30 complete interview sessions of six questions each, including their follow-ups. Table 3.5 shows the results.

Table 3.5: API response times in milliseconds on a local deployment

Endpoint	Calls	Mean	Median	95th pct.
POST /api/auth/guest	30	92.2	90.6	110.7
POST /api/interviews	30	12.0	11.1	15.5
POST /api/interviews/:id/answers	270	12.4	11.9	17.2
POST /api/interviews/:id/complete	30	10.7	10.4	14.0
GET /api/interviews/:id	32	5.9	5.8	8.5
GET /api/stats	30	6.0	5.8	8.0
POST /api/practice/questions	30	3.5	3.5	4.7

Starting a profile is the slowest call because it hashes a new device key with bcrypt, which is deliberately expensive. All interview operations completed in about 12 ms on average, and the debrief was ready on average 7.7 ms after the interview was finished (33 ms at the 95th percentile), so with the offline engine the candidate never waits. With Claude enabled, generation and evaluation take longer, which is why evaluation runs in the background and the interface shows live progress.

Finally, all automated tests were run against a local MongoDB server so that the API integration tests were included. Table 3.6 summarises the results:

all 40 tests passed, and the type-check and the production build of all three packages also completed without errors.

Table 3.6: Automated test results

Test suite	What it covers	Tests	Result
shared/src/ analysis.test.ts	Fillers and hedges, “like” disambiguation, pace, sentence estimation, STAR detection, annotation	9	Passed
server/test/engine.test.ts	Question bank integrity, selection and tailoring, scoring order, feedback, follow-ups, skills extraction	12	Passed
server/test/api.test.ts	Browser profiles, authentication, validation, interview lifecycle, ordering, ownership, Question Lab, demo, deletion	19	Passed
Total		**40**	**40 passed**

In summary, the results show that:

- All 40 automated tests passed, together with the type-check and the production builds of all three packages.
- The offline scorer ranked model, partial and vague answers in the correct order for all 115 bank questions.
- 85 per cent of model answers scored above 80 and 99 per cent of off-topic answers scored below 35.
- Offline evaluation took less than 0.3 ms per answer, and interview API calls completed in about 12 ms on average.

3.7 Conclusion

This project designed and implemented Poise, a full-stack AI-based interview practice and feedback system. Built on the MERN stack in TypeScript, it runs realistic spoken mock interviews in the browser, generates questions for ten roles and four formats tailored to a job description, asks

adaptive follow-up questions, and evaluates every answer on relevance, depth, structure, clarity and confidence with specific, evidence-based feedback and a stronger sample answer. Each session ends with a debrief and a practice plan, and the dashboard and history make progress visible over time.

The pluggable AI layer is the central design decision. Claude provides fluent, context-aware questions and feedback through schema-validated structured outputs, while the offline engine, with its curated bank of 115 questions and transparent scoring rubric, guarantees that the application always works, even without internet access, and gives scores that can be explained. The evaluation confirmed that the system behaves as intended: all 40 automated tests passed, the offline scorer ordered answers of different quality correctly for every question in the bank, and interview operations completed in about 12 ms on a local deployment. Privacy and security were treated as requirements, with no sign-up, hashed device keys, httpOnly cookies, input validation, rate limiting, and audio and video processed only in the browser.

Several enhancements can extend the system further: coding rounds with an editor and automatic test execution for data-structure and algorithm questions; résumé upload to tailor behavioural questions to the candidate's own projects; server-side speech-to-text, for example with Whisper, to support browsers without the Web Speech API and to capture “um” and “uh”, which some browsers drop from transcripts; and a peer or mentor review mode with shared reports.

APPENDIX

Source Code

```
// shared/src/analysis.ts — delivery metrics for one answer
export function analyzeAnswer(
  text: string,
  options: AnalyzeOptions = {},
): AnswerMetrics {
  const tokens = tokenize(text);
  const words = tokens.filter((t) => t.word.length > 0);
  const kinds = classifyTokens(tokens);
  const fillers = phraseCounts(tokens, kinds, "filler");
  const hedges = phraseCounts(tokens, kinds, "hedge");
  const wordCount = words.length;
  const sentenceCount = countSentences(text, wordCount);
  const durationSec =
    Math.max(0, Math.round(options.durationSec ?? 0));
  const wpm =
    options.mode === "voice" && durationSec >= 10 && wordCount > 0
      ? Math.round(wordCount / (durationSec / 60))
      : null;
  const normalized = normalize(text);
  return {
    wordCount,
    durationSec,
    wpm,
    fillerCount: fillers.total,
```

```
    fillerWords: fillers.counts,
    hedgeCount: hedges.total,
    hedgeWords: hedges.counts,
    sentenceCount,
    avgSentenceLength: sentenceCount > 0
      ? Math.round((wordCount / sentenceCount) * 10) / 10 : 0,
    star: detectStar(text),
    hasNumbers: NUMBER_WORDS.test(normalized),
    hasExample:
      EXAMPLE_MARKERS.some((m) => normalized.includes(m)),
  };
}
// server/src/ai/local/scoring.ts — rubric-based offline scoring
export function scoreAnswer(input: ScoringInput): ScoringResult {
  const { metrics, answer } = input;
  const words = metrics.wordCount;
  const normalized = normalize(answer);
  const match = matchKeyPoints(answer, input.points);
  const overlap = questionOverlap(input.question, answer);
  const connectors = countOccurrences(normalized, CONNECTORS);
  const mentionsTradeoffs =
    TRADEOFF_MARKERS.some((m) => normalized.includes(m));
  const assertive =
    countOccurrences(normalized, ASSERTIVE_MARKERS);
  const fillersPer100 =
    words > 0 ? (metrics.fillerCount / words) * 100 : 0;
  const hedgesPer100 =
```

```
  words > 0 ? (metrics.hedgeCount / words) * 100 : 0;
const signals = { connectors, mentionsTradeoffs, assertive,
  fillersPer100, hedgesPer100, overlap };
if (words === 0) {
  return {
    scores: { relevance: 0, depth: 0, structure: 0,
      clarity: 0, confidence: 0 },
    match,
    signals,
  };
}
const coverage = input.points.length > 0
  ? match.coverage : Math.min(1, overlap * 1.4);
// Relevance — did the answer address what was asked?
let relevance =
  20 + 55 * coverage + 25 * Math.min(1, overlap / 0.45);
if (words < 12) relevance = Math.min(relevance, 30);
else if (words < 25) relevance = Math.min(relevance, 55);
// Depth — substance, specifics and coverage of the key ideas.
const lengthScore =
  words < 20 ? 15 : words < 40 ? 35 : words < 70 ? 55
  : words < 110 ? 72 : words <= 260 ? 85 : words <= 380 ? 78 : 65;
const specifics = (metrics.hasNumbers ? 8 : 0)
  + (metrics.hasExample ? 8 : 0) + (mentionsTradeoffs ? 6 : 0);
const depth = 0.45 * lengthScore + 40 * coverage + specifics;
// Structure — STAR for stories, signposting for explanations.
let structure: number;
```

```
if (input.behavioral) {
  const parts = Object.values(metrics.star).filter(Boolean).length;
  structure =
    25 + parts * 17 + (metrics.sentenceCount >= 4 ? 7 : 0);
} else {
  const firstSentence =
    normalize(answer.split(/[.!?]/)[0] ?? "");
  const direct =
    questionOverlap(input.question, firstSentence) >= 0.2 ? 10 : 0;
  const sentences = metrics.sentenceCount >= 3 ? 15
    : metrics.sentenceCount >= 2 ? 8 : 0;
  structure =
    35 + Math.min(36, connectors * 6) + sentences + direct;
}
if (words < 25) structure = Math.min(structure, 40);
// Clarity — concise sentences, little noise.
let clarity = 92;
if (metrics.avgSentenceLength > 30)
  clarity -= Math.min(25, (metrics.avgSentenceLength - 30) * 1.5);
if (metrics.avgSentenceLength < 6 && metrics.sentenceCount > 2)
  clarity -= 10;
clarity -= Math.min(30, fillersPer100 * 5);
if (words < 15) clarity = Math.min(clarity, 60);
// Confidence — decisive language, steady pace.
let confidence = 86;
confidence -= Math.min(30, hedgesPer100 * 6);
confidence -= Math.min(20, fillersPer100 * 3);
```

```
if (metrics.wpm !== null) {
  if (metrics.wpm < 100)
    confidence -= Math.min(15, (100 - metrics.wpm) * 0.4);
  if (metrics.wpm > 175)
    confidence -= Math.min(15, (metrics.wpm - 175) * 0.4);
}
confidence += Math.min(8, assertive * 4);
if (words < 25) confidence = Math.min(confidence, 50);
// An answer that touches none of the key ideas and barely overlaps
// with the question is off-topic: fluent delivery shouldn't rescue it.
if (input.points.length > 0 && match.coverage === 0
    && overlap < 0.15) {
  structure = Math.min(structure, 30);
  clarity = Math.min(clarity, 40);
  confidence = Math.min(confidence, 40);
}
return {
  scores: {
    relevance: clamp(relevance),
    depth: clamp(depth),
    structure: clamp(structure),
    clarity: clamp(clarity),
    confidence: clamp(confidence),
  },
  match,
  signals,
};
```

```
}
// server/src/ai/index.ts — engine router with automatic fallback
export async function runEngine<T>(
  operation: string,
  task: (engine: InterviewEngine) => Promise<T>,
): Promise<{ value: T; engine: EngineKind }> {
  if (claude) {
    try {
      return { value: await task(claude), engine: "claude" };
    } catch (error) {
      logger.warn(`Claude ${operation} failed `
        + `(${describeError(error)}); `
        + "using the offline engine instead.");
    }
  }
  return { value: await task(local), engine: "local" };
}
// server/src/services/interview.service.ts — recording an answer
export async function submitAnswer(
  userId: string, sessionId: string, input: AnswerInput,
): Promise<InterviewSession> {
  const session = await loadOwned(userId, sessionId);
  if (session.status !== "in_progress")
    throw conflict("This interview has already finished.");
  const index = session.questions
    .findIndex((q) => q.id === input.questionId);
  const question = session.questions[index];
```

```
if (!question)
  throw notFound("That question isn't part of this interview.");
if (question.answer)
  throw conflict("You've already answered this question.");
if (index !== session.currentIndex)
  throw conflict("Please answer the questions in order.");
const text = input.skipped ? "" : input.text.trim();
if (!input.skipped && text.length === 0)
  throw badRequest("Your answer is empty — type or speak an "
    + "answer, or skip the question.");
const durationSec = Math.max(0,
  Math.min(Math.round(input.durationSec), 60 * 60));
const metrics =
  analyzeAnswer(text, { durationSec, mode: input.mode });
const answer = { text, durationSec, mode: input.mode,
  skipped: input.skipped, submittedAt: new Date().toISOString() };
const pending: QuestionEvaluation = { status: "pending",
  strengths: [], improvements: [], missedPoints: [] };
const saved = await InterviewSessionModel.updateOne(
  { _id: sessionId, user: userId, status: "in_progress",
    currentIndex: index, "questions.id": question.id },
  {
    $set: {
      "questions.$.answer": answer,
      "questions.$.metrics": metrics,
      "questions.$.evaluation": pending,
      currentIndex: index + 1,
```

```
    },
    $inc: { totalDurationSec: durationSec },
  },
);
if (saved.modifiedCount === 0)
  throw conflict("This answer was already recorded.");
scheduleEvaluation(sessionId, question.id);
// Adaptive follow-up: probe the answer like a real interviewer would.
const followUps = session.questions
  .filter((q) => q.kind === "follow-up").length;
const maxFollowUps = Math.floor(
  session.config.questionCount * LIMITS.maxFollowUpsRatio);
if (session.config.adaptiveFollowUps && !input.skipped
    && question.kind === "core" && followUps < maxFollowUps) {
  try {
    const { value: followUp } = await runEngine("follow-up",
      (e) => e.generateFollowUp({
        config: session.config,
        question: contextOf(question),
        answer: text,
        metrics,
        askedQuestions: session.questions.map((q) => q.text),
      }),
    );
    if (followUp) {
      await InterviewSessionModel.updateOne({ _id: sessionId }, {
        $push: { questions: {
```

```
          $each: [toStored(followUp, "follow-up", question.id)],
          $position: index + 1 } },
        });
      }
    } catch (error) {
      logger.warn(
        "Follow-up generation failed; continuing without one.", error);
    }
  }
  return serializeSession(await loadOwned(userId, sessionId));
}
// client/src/hooks/useSpeechRecognition.ts — continuous dictation
const recognition = new Recognition();
recognition.continuous = true;
recognition.interimResults = true;
recognition.lang = lang;
recognition.onresult = (event) => {
  let pending = "";
  for (let i = event.resultIndex; i < event.results.length; i++) {
    const result = event.results[i]!;
    const text = result[0]?.transcript ?? "";
    if (result.isFinal) onFinalRef.current(text.trim());
    else pending += text;
  }
  setInterim(pending.trim());
};
recognition.onerror = (event) => {
```

```
  if (event.error === "no-speech" || event.error === "aborted") return;
  setError(ERROR_MESSAGES[event.error] ?? "Speech recognition "
    + "stopped unexpectedly. Try again, or type your answer.");
  wantListening.current = false;
};
// Browsers stop listening after a pause, so it restarts itself
// until stop() is called.
recognition.onend = () => {
  setInterim("");
  if (wantListening.current) {
    try {
      recognition.start();
      return;
    } catch {
      /* already started */
    }
  }
  setListening(false);
};
```